Лабораторна робота №6

З дисципліни “Системи програмування”

На тему:“Файлове введення-виведення. Рекурсивна навігація файловою системою та розробка графічного інтерфейсу“

Студента 4 курсу: Групи
МІТ-41 Демиденко Андрій

Київ - 2026р.

1. МЕТА РОБОТИ

Дослідити підсистему вводу-виводу платформи .NET (простір імен `System.IO`). Опанувати роботу з класами `DirectoryInfo` та `FileInfo`. Навчитися реалізовувати алгоритми рекурсивного обходу файлової системи, отримання метаданих об'єктів та розробити графічний кросплатформний файловий менеджер, адаптований для середовища GNU/Linux (Debian).

2. ЗАВДАННЯ 1 ТА 2: ІЄРАРХІЧНИЙ ОБХІД ТА ПОШУК (CLI)

Опис реалізації: Перші два завдання реалізовані у вигляді консольного застосунку. Для взаємодії з файловою системою використано абстракції `DirectoryInfo` та `FileInfo`. В основі лежить алгоритм пошуку в глибину (Depth-First Search, DFS), що реалізований через рекурсію.

- **Завдання 1:** Метод `PrintDirectoryTree` рекурсивно обходить каталоги, генеруючи псевдографічні відступи для візуалізації рівнів вкладеності (дерево).
- **Завдання 2:** Метод `SearchFile` приймає шаблон імені та використовує фільтрацію на рівні системного виклику (`GetFiles(fileName)`), що значно оптимізує використання оперативної пам'яті.

Специфіка операційної системи (Linux): Оскільки програма виконується в ОС Debian Linux, діє сувора модель прав доступу POSIX ACL. Для запобігання аварійному завершенню програми при спробі прочитати системні або чужі каталоги, у рекурсивні методи імплементовано "глухе" перехоплення винятку `UnauthorizedAccessException`.

```
=== ЛАБОРАТОРНА РОБОТА 6: ФАЙЛОВЕ ВВЕДЕННЯ-ВИВЕДЕННЯ ===

[ЗАВДАННЯ 1]: Ієрархія директорії: /home/marino/My pc/4 курс/System programing

└── [System programing]
    ├── .gitignore
    ├── [Lab1]
    │   ├── Task2.cs
    │   ├── Task3.cs
    │   └── Task1.cs
    ├── [Lab5]
    │   ├── app.manifest
    │   ├── DynamicTypeIdentification.csproj
    │   ├── Program.cs
    │   ├── MainWindow.axaml.cs
    │   ├── MainWindow.axaml
    │   ├── App.axaml.cs
```

Рисунок 1 — Ієрархічне відображення директорій та рекурсивний пошук файлу в консолі.

3. ЗАВДАННЯ 3: ГРАФІЧНИЙ ФАЙЛОВИЙ МЕНЕДЖЕР (AVALONIA UI)

Опис реалізації: Для виконання третього завдання був ініційований новий проект на базі кросплатформного фреймворку **Avalonia UI**, оскільки класичні WinForms/WPF не підтримуються нативно в Linux.

- **Інтерфейс (XAML):** Створено головне вікно з панеллю навігації (кнопка "Вгору", адресний рядок `TextBox`, кнопка "Перейти") та основним компонентом `ListBox` для виведення вмісту. Використано строго типізований `DataTemplate` (Compiled Bindings) для форматування відображення (Іконка, Назва, Розмір, Дата модифікації).
- **Модель даних:** Розроблено клас `FileSystemItem`, що інкапсулює метадані та використовує ініціалізацію за замовчуванням (уникнення `NullReferenceException`).
- **Бізнес-логіка:** Метод `LoadDirectory` зчитує вміст шляху, застосовуючи жорстку фільтрацію прихованих файлів Linux (ті, що починаються з крапки `StartsWith(".")`) та системних файлів. Стан директорії зберігається в `ObservableCollection`, що забезпечує автоматичне перемальовування інтерфейсу при зміні папки.

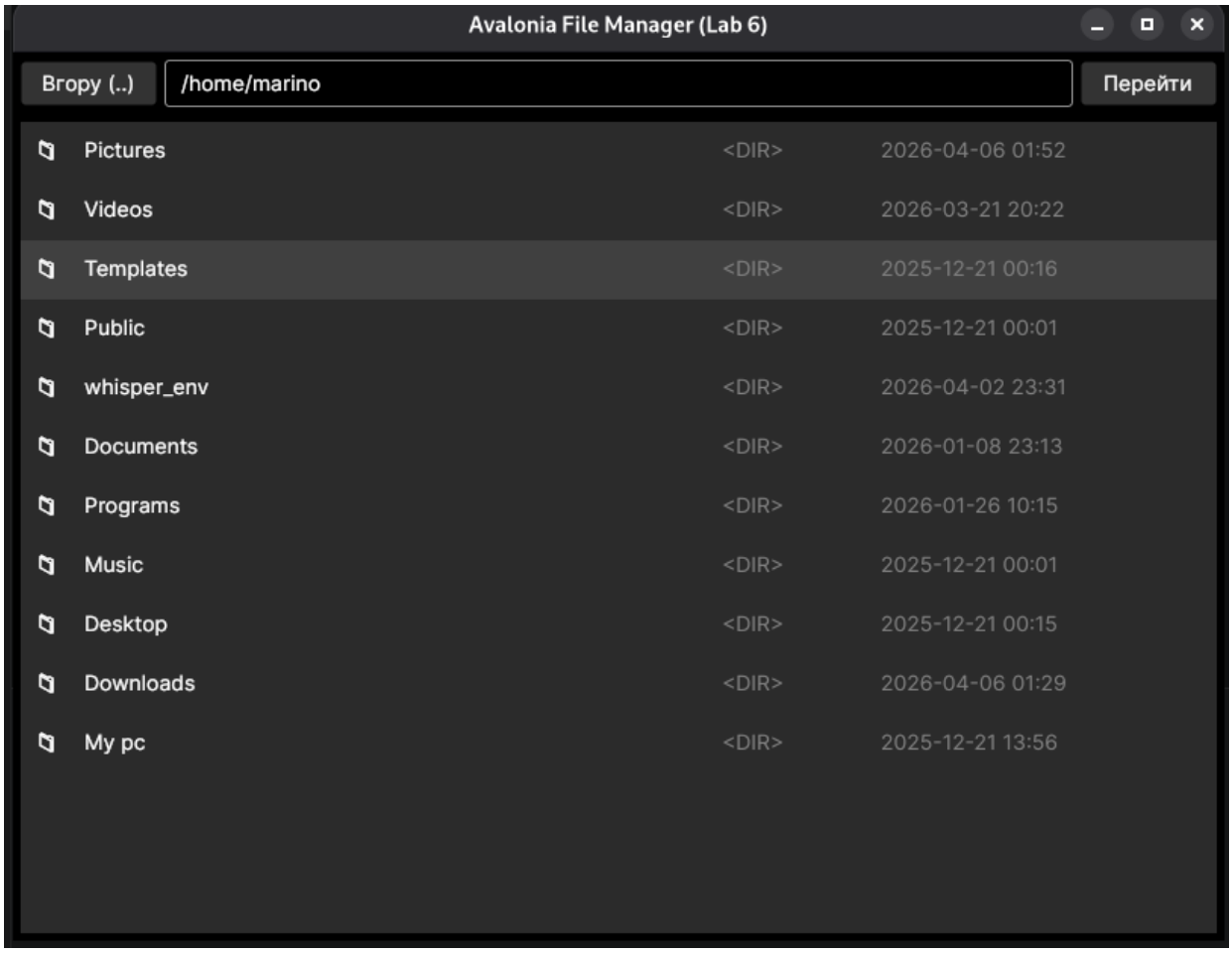

Рисунок 2 — Графічний інтерфейс розробленого кросплатформного файлового менеджера.

4. КОНТРОЛЬНІ ПИТАННЯ

Питання 1: У чому різниця між класами `File/Directory` та `FileInfo/DirectoryInfo`?
Відповідь: Класи `File` та `Directory` містять виключно статичні методи. Вони зручні для швидких, одноразових операцій (наприклад, `File.ReadAllText`), але при кожному виклику вони виконують перевірку прав доступу до файлу. Класи `FileInfo` та `DirectoryInfo` вимагають створення екземпляра (інстанціювання). Перевірка безпеки виконується лише один раз під час створення об'єкта, після чого можна багаторазово і швидко звертатися до його властивостей (розмір, дата), що робить їх набагато ефективнішими при обході тисяч файлів.

Питання 2: Яка алгоритмічна проблема існує при використанні методу `GetFiles()` для великих директорій і як її вирішити?

Відповідь: Метод `GetFiles()` спочатку зчитує абсолютно всі файли директорії, виділяє під них пам'ять (створює масив `FileInfo[]`), і лише потім повертає результат. Якщо в папці сотні тисяч файлів, це спричинить різкий стрибок використання RAM і блокування (зависання) потоку. Оптимальним рішенням у сучасних версіях .NET є використання методу `EnumerateFiles()`. Він використовує відкладене виконання (ітератор) і повертає файли по одному по мірі їх знаходження, забезпечуючи O(1) просторову складність.

ВИСНОВОК

У ході виконання лабораторної роботи №6 було детально досліджено інфраструктуру платформи .NET для роботи з файловою системою (`System.IO`). На практиці було реалізовано рекурсивні алгоритми пошуку в глибину для обходу ієрархічних структур (дерев каталогів) із коректною обробкою системних прав доступу в середовищі Linux.

Розробка графічного файлового менеджера довела ефективність використання кросплатформного фреймворку Avalonia UI. Застосування патерну Data Binding разом із колекцією `ObservableCollection` дозволило створити чуйний графічний інтерфейс, який динамічно відображає поточний стан файлової системи, а строга типізація шаблонів забезпечила високу швидкість рендерингу великих списків файлів.